Module: `Problem 6`

> This module provides two simple, in-place sorting utilities that demonstrate elementary element-wise swapping using nested loops. Both functions mutate the supplied list directly, output the list's state after each swap, and return `None`. They are primarily educational examples of quadratic-time sorting algorithms.

Functions

`swapSort(L)`

Description

Performs an in-place sort of `L` by iterating over each element `i` and swapping it with any later element `j` that is smaller. The algorithm is a naïve selection-sort variant with a time complexity of $O(n^2)$.

Parameters

- `L` (*list* of *comparable items*): The list to be sorted. Although the docstring mentions integers, the implementation works with any objects that support the `<` comparison operator (e.g., strings).

Returns

`None`. The list `L` is sorted **in place**; the function does not produce a return value.

Examples

```
# Example 1 - integer list
result = swapSort([1, 2, 3])   # → None
# Console output:
# Original L:  [1, 2, 3]
# Final L:  [1, 2, 3]

# Example 2 - list of strings
result = swapSort(['a', 'b', 'c'])   # → None
# Console output:
# Original L:  ['a', 'b', 'c']
# Final L:  ['a', 'b', 'c']

# Example 3 - single-element list
result = swapSort(['test'])   # → None
# Console output:
# Original L:  ['test']
# Final L:  ['test']
```

Edge Cases / Notes - The function prints the original list, each intermediate state after a swap, and the final sorted list. These side-effects are useful for tracing but may be undesirable in production code. - Because it only compares `L[j] < L[i]` for `j > i`, the algorithm never swaps an element with itself. - If the list contains elements that are not mutually comparable (e.g., mixing integers and strings), a `TypeError` will be raised at the comparison step.

`modSwapSort(L)`

Description

A modified version of `swapSort` that iterates over **all** pairs of indices `(i, j)`, including cases where `i == j`. Whenever it finds `L[j] < L[i]`, it swaps the two elements. This also results in a sorted list but performs many redundant comparisons and swaps, still operating in *O(n²)* time.

Parameters

- `L` (*list* of *comparable items*): The list to be sorted. Like `swapSort`, it works with any elements that support the `<` operator.

Returns

`None`. The list `L` is sorted **in place**; no value is returned.

Examples

```
# Example 1 - integer list in reverse order
result = modSwapSort([3, 2, 1])   # → None
# Console output (excerpt):
# Original L:  [3, 2, 1]
# [2, 3, 1]
# [1, 3, 2]
# [1, 2, 3]
# Final L:  [1, 2, 3]

# Example 2 - list of strings in reverse order
result = modSwapSort(['c', 'b', 'a'])   # → None
# Console output (excerpt):
# Original L:  ['c', 'b', 'a']
# ['b', 'c', 'a']
# ['a', 'c', 'b']
# ['a', 'b', 'c']
# Final L:  ['a', 'b', 'c']

# Example 3 - single-element list
result = modSwapSort(['test'])   # → None
# Console output:
# Original L:  ['test']
# Final L:  ['test']
```

Edge Cases / Notes - The inner loop runs from `0` to `len(L) - 1` for every `i`, causing unnecessary self-comparisons (`i == j`). When `i == j`, the condition `L[j] < L[i]` is never true, so no swap occurs, but the extra iterations add overhead. - Like `swapSort`, the function prints each intermediate list state, which can be noisy for large inputs. - The algorithm assumes that all elements are mutually comparable; otherwise, a `TypeError` will be raised during the `<` comparison.
